

Vite & Gourmand Documentation Technique

Architecture, MCD, Diagrammes UML, API, Sécurité, Déploiement

Version 1.2 — 2026

ECF Studi — TP Développeur Web et Web Mobile

Élément	Lien
Application	https://vitegourmand33.vercel.app
API Backend	https://vite-gourmand-back-chap.onrender.com
GitHub Front	https://github.com/NicolasVrignaudVG/Vite-Gourmand
GitHub Back	https://github.com/NicolasVrignaudVG/Vite-Gourmand-back

1. Architecture de l'application

1.1 Vue d'ensemble

Vite & Gourmand est une application web full-stack à architecture découplée : le frontend (SPA JavaScript vanilla) et le backend (API REST Symfony) sont deux projets distincts, déployés séparément et communiquant via HTTP/JSON. L'authentification est assurée par des tokens JWT transportés dans des cookies HttpOnly. En production, un proxy Vercel réécrit les appels `/api/*` du frontend vers le backend Render, de sorte que le navigateur ne voit qu'un seul domaine (voir section 4.3).

Couche	Technologie	Hébergement
Frontend	HTML / CSS / JS Vanilla (SPA)	Vercel
Backend API	PHP 8.4 + Symfony 8	Render (Docker)
Base SQL	MySQL 8.0	Clever Cloud
Base NoSQL	MongoDB Atlas (M0)	MongoDB Cloud
E-mails	Brevo API	Brevo (SaaS)
Livraison	OpenRouteService API	ORS Cloud

1.2 Stack technique

Technologie	Rôle
PHP 8.4	Langage backend — typage strict, attributs natifs, enums
Symfony 8	Framework PHP — Routing, Security, Serializer, Mailer
Doctrine ORM	Mapping objet-relationnel — Migrations, Repositories
LexikJWTAuthenticationBundle	Authentification JWT (RS256, clés RSA 4096 bits)
NelmioApiDocBundle	Documentation OpenAPI / Swagger UI (<code>/api/doc</code>)
NelmioSecurityBundle	En-têtes de sécurité HTTP + Content Security Policy
Extension MongoDB PHP	Connexion à MongoDB Atlas pour les statistiques
JavaScript ES6+	Frontend vanilla — fetch API, modules, async/await
Docker	Conteneurisation du backend (PHP 8.4 + Apache)
Déploiement continu	Webhooks Git natifs Vercel & Render (push main → deploy)

2. Modèle Conceptuel de Données

2.1 Entités principales

La base relationnelle MySQL comprend 18 tables : 14 entités métier et 4 tables d'association (commande_menu, commande_plat, menu_plat, plat_allergene). Les principales entités et leurs attributs :

Table utilisateur	Type	Description
id	INT PK	Identifiant unique auto-incrémenté
email	VARCHAR(255)	Adresse e-mail unique — login
password	VARCHAR(255)	Mot de passe haché bcrypt (coût 12)
nom / prenom	VARCHAR(100)	Identité du client
telephone / adresse	VARCHAR	Coordonnées (nullable)
pseudonyme	VARCHAR(50)	Nom public affiché avec les avis (minimisation RGPD — à défaut, « Client vérifié »)
actif	TINYINT(1)	Compte actif (1) ou désactivé (0)
role_id	INT FK	Référence vers la table role
created_at	DATETIME	Date de création du compte

Table commande	Type	Description
id	INT PK	Identifiant unique
numero_commande	VARCHAR(20)	Référence unique ex: VG-A1B2C3
utilisateur_id	INT FK	Client ayant passé la commande
date_prestation	DATETIME	Date et heure souhaitées
adresse/ville/cp_livraison	VARCHAR	Lieu de livraison
nombre_personnes	INT	Nombre total de personnes
prix_menu / prix_livraison	DECIMAL	Détails tarifaires
prix_total / remise	DECIMAL	Total TTC et remise appliquée
statut	VARCHAR(20)	Statut courant (voir machine à états)
pret_materiel	TINYINT(1)	Matériel prêté au client

Les tables commande_menu et commande_plat sont des tables d'association porteuses d'attributs (nombre de personnes, prix, remise par menu), permettant les commandes multi-menus. La table suivi_commande historise chaque changement de statut. La table refresh_token gère la rotation des jetons de rafraîchissement.

2.2 Relations principales

Entité 1	Card.	Entité 2	Description
utilisateur	1-N	commande	Un utilisateur passe plusieurs commandes
utilisateur	N-1	role	Un utilisateur possède exactement un rôle
utilisateur	1-N	refresh_token	Rotation des jetons de rafraîchissement
menu	1-N	menu_image	Un menu a plusieurs images

Entité 1	Card.	Entité 2	Description
menu	N-1	theme / regime	Thématique et régime (nullable)
menu	N-N	plat	Composition via menu_plat
commande	N-N	menu	Multi-menus via commande_menu
commande	1-N	suivi_commande	Historique des statuts
commande	1-N	commande_plat	Plats choisis
plat	N-N	allergene	Allergènes via plat_allergene
utilisateur	1-N	avis	Avis rédigés (liés à une commande)

2.3 MCD (Merise)

Le schéma ci-dessous suit la convention Merise : chaque cardinalité est portée du côté de l'entité dont elle décrit la participation (ex. UTILISATEUR (1,1) — possède — ROLE (0,n) : un utilisateur possède exactement un rôle, un rôle peut être porté par plusieurs utilisateurs).

MCD - Vite & Gourmand (v2 - cardinalités corrigées)

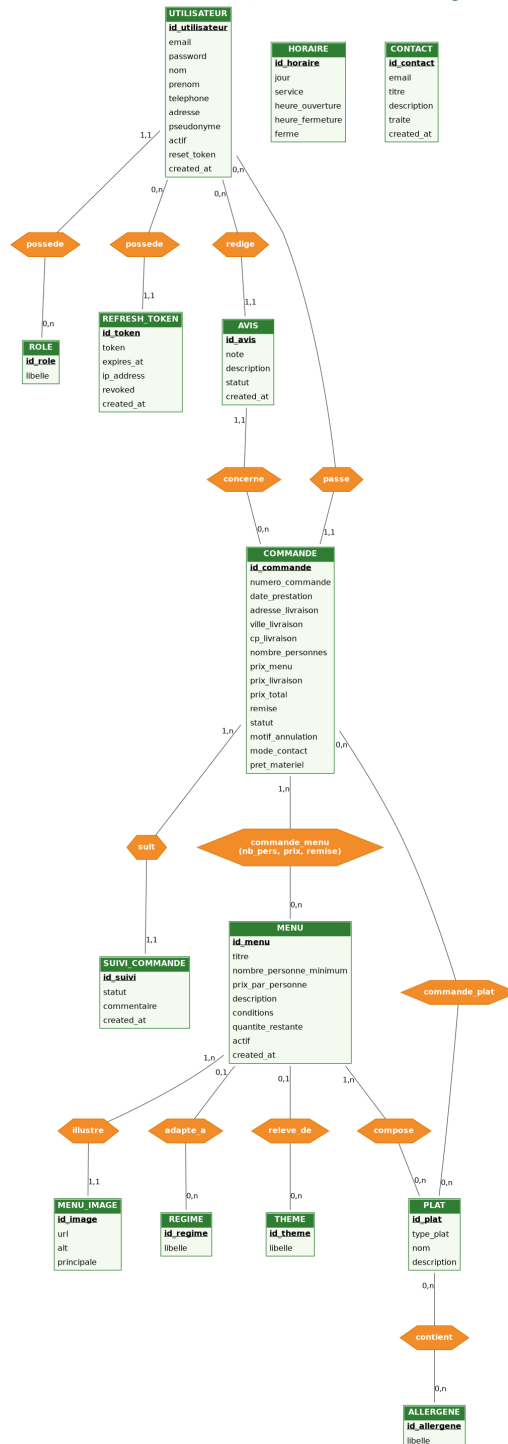


Figure 1 — Modèle Conceptuel de Données (v2)

2.4 Diagramme de cas d'utilisation

Quatre acteurs interagissent avec le système. L'héritage suit la hiérarchie des rôles : le client hérite des cas du visiteur, l'employé de ceux du client, l'administrateur de ceux de l'employé. Les relations «include» matérialisent l'authentification obligatoire, les relations «extend» les cas optionnels.

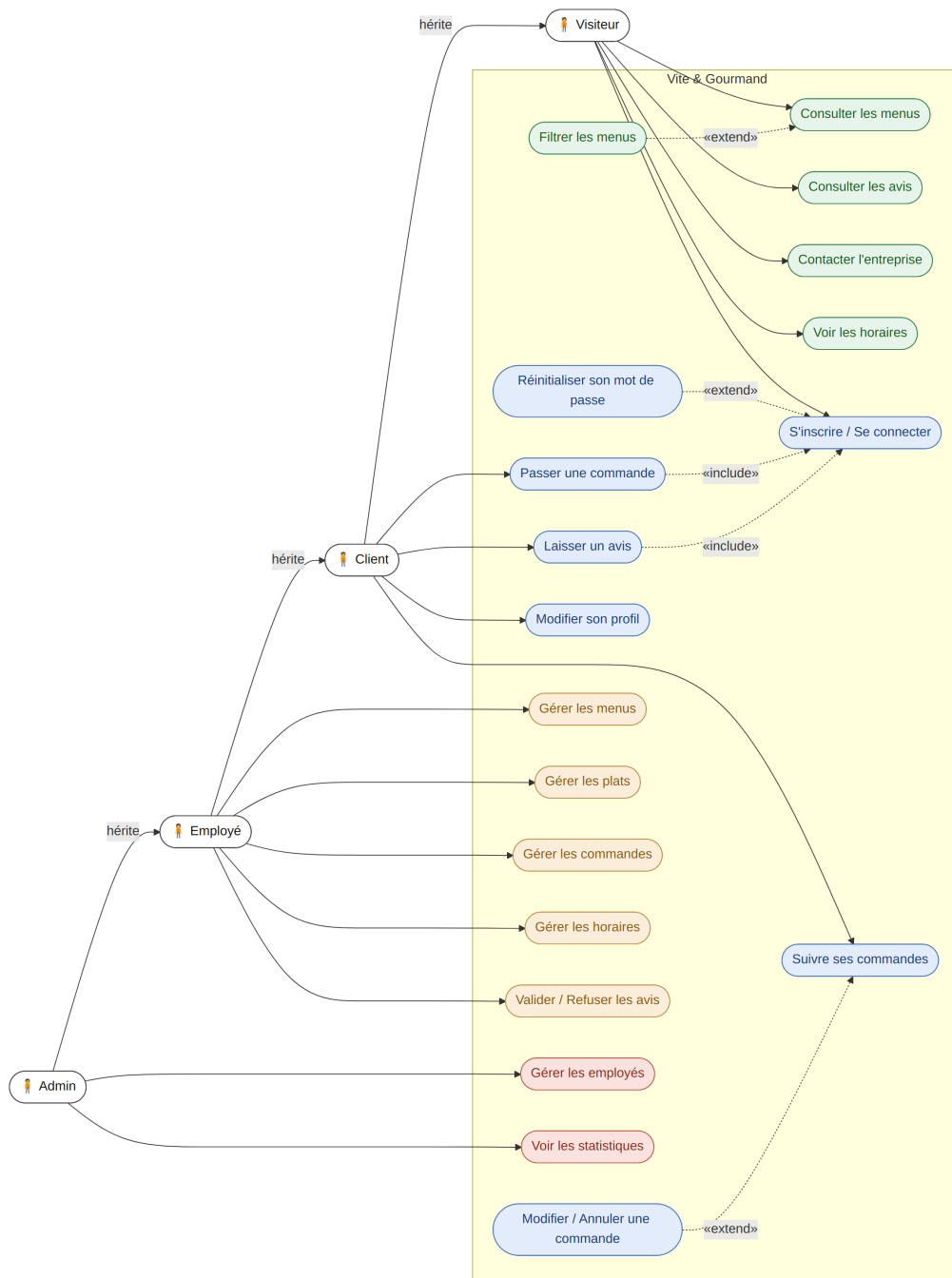


Figure 2 — Diagramme de cas d'utilisation

2.5 Diagramme de séquence — cycle de vie JWT

Le diagramme couvre les quatre phases du cycle d'authentification : connexion (pose des deux cookies HttpOnly), requête authentifiée, renouvellement avec rotation du refresh token (révocation de l'ancien jeton en base), et déconnexion (révocation + suppression des cookies).

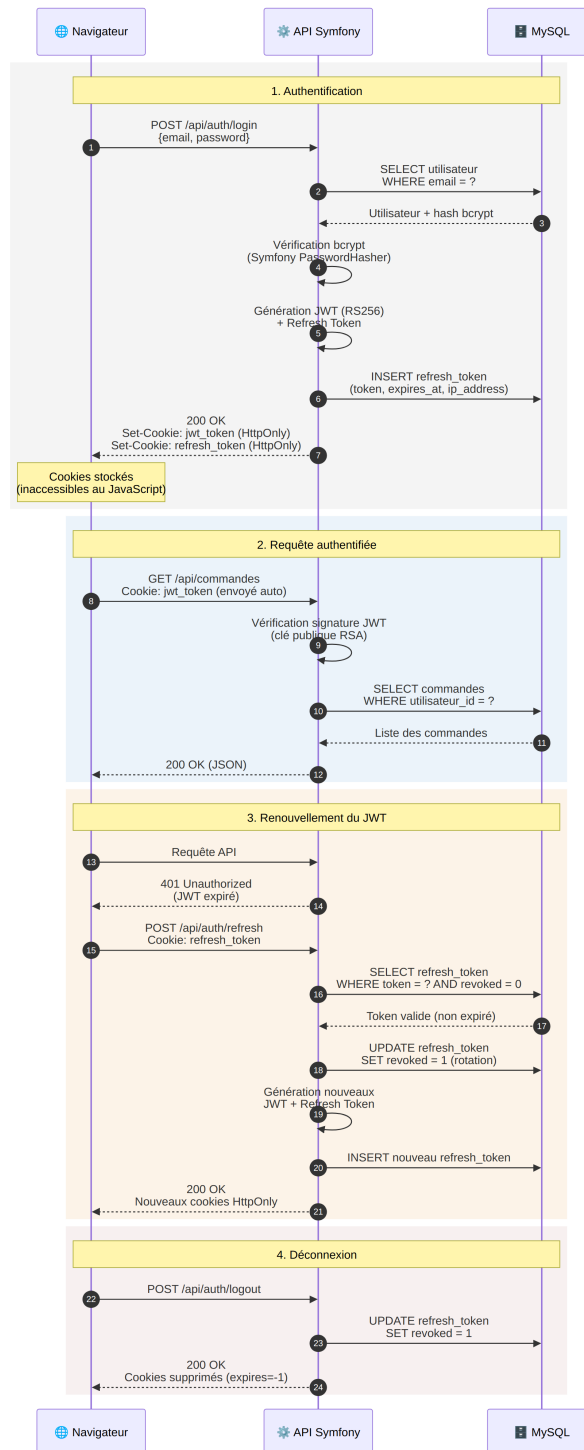


Figure 3 — Séquence authentification JWT (rotation des refresh tokens)

3. API REST

3.1 Authentification

L'API utilise JWT (RS256, clés RSA 4096 bits). Le token est stocké dans un cookie HttpOnly (non accessible au JavaScript, protection contre le vol par XSS), accompagné d'un refresh token à rotation. La connexion pose automatiquement les cookies.

Méthod e	Route	Auth	Description
POST	/api/auth/register	Public	Créer un compte utilisateur
POST	/api/auth/login	Public	Connexion — pose les cookies JWT
POST	/api/auth/refresh	Cookie	Renouvelle le JWT via refresh token
POST	/api/auth/logout	Cookie	Déconnexion — supprime les cookies
GET	/api/auth/me	JWT	Profil de l'utilisateur connecté
PUT	/api/auth/me	JWT	Modifier son profil
DELETE	/api/auth/me	JWT	Supprimer son compte (RGPD)
POST	/api/auth/forgot-password	Public	Demande de réinitialisation
POST	/api/auth/reset-password	Public	Réinitialiser via token e-mail

3.2 Menus & Commandes

Méthod e	Route	Auth	Description
GET	/api/menus	Public	Liste (filtres: thème, régime, prix, personnes)
GET	/api/menus/{id}	Public	Détail d'un menu (plats, images)
POST	/api/menus	Employ é	Créer un menu
PUT	/api/menus/{id}	Employ é	Modifier un menu
DELETE	/api/menus/{id}	Employ é	Désactiver (soft delete)
GET	/api/commandes	JWT	Mes commandes
POST	/api/commandes	JWT	Créer une commande (multi-menus)
PUT	/api/commandes/{id}	JWT	Modifier (si statut en_attente)
DELETE	/api/commandes/{id}	JWT	Annuler (si statut en_attente)
GET	/api/commandes/toutes	Employ é	Toutes les commandes
PATCH	/api/commandes/{id}/statut	Employ é	Changer le statut

3.3 Administration & autres

Méthod e	Route	Auth	Description
GET	/api/admin/employees	Admin	Liste des employés
POST	/api/admin/employees	Admin	Créer un compte employé
PATCH	/api/admin/employees/{id}/toggle	Admin	Activer / désactiver
GET	/api/admin/stats	Admin	Statistiques MongoDB (filtres menu, dates)
GET	/api/avis	Public	Avis validés publics
PATCH	/api/avis/{id}/valider	Employé	Valider un avis
PATCH	/api/avis/{id}/refuser	Employé	Refuser un avis
POST	/api/contact	Public	Envoyer un message de contact
GET	/api/doc	Public	Documentation Swagger UI

4. Sécurité

4.1 Authentification JWT

Aspect	Détail
Algorithme	RS256 — clés asymétriques RSA 4096 bits
Durée de vie	Token courte durée + refresh token à rotation
Stockage client	Cookie HttpOnly (inaccessible au JavaScript)
Protection XSS	Le token n'étant pas en localStorage, il ne peut être volé par script
Refresh token	Rotation à chaque renouvellement + révocation en base
Cookies	SameSite=Lax + Secure (même domaine grâce au proxy Vercel)
Clés	private.pem / public.pem — hors Git, en base64 dans Render

■ *Choix d'architecture : le stockage du JWT en cookie HttpOnly (plutôt qu'en localStorage) protège contre l'exfiltration du token par injection de script (XSS). La contrainte cross-origin initiale entre le front (Vercel) et le back (Render) a été levée par la mise en place d'un proxy Vercel (voir 4.3) : le navigateur n'échange qu'avec un seul domaine.*

4.2 Contrôle d'accès (RBAC)

Rôle	Périmètre
ROLE_USER	Utilisateur authentifié — commandes, profil, avis
ROLE_EMPLOYE	Hérite de USER — gestion menus, commandes, avis, horaires
ROLE_ADMIN	Hérite de EMPLOYE — gestion employés, statistiques

Le contrôle est appliqué via l'attribut `#[IsGranted('ROLE_X')]` sur les routes, et une hiérarchie de rôles définie dans `security.yaml`.

4.3 Cookies cross-origin et protection CSRF

Le front (Vercel) et le back (Render) étant initialement sur deux domaines distincts, les cookies devaient être émis en `SameSite=None`, une configuration fragilisée par le blocage des cookies tiers de Chrome 120+. Un proxy a donc été mis en place dans `vercel.json` : les requêtes `/api/*` émises vers le domaine du front sont réécrites côté serveur vers l'API Render. Du point de vue du navigateur, front et API partagent le même domaine, ce qui a permis de passer les cookies en `SameSite=Lax + Secure` — plus restrictif et plus sûr contre le CSRF.

La protection CSRF repose ainsi sur trois éléments combinés : l'attribut `SameSite=Lax` des cookies, la politique CORS restrictive (seule l'origine du front de production est autorisée pour les accès directs à l'API), et l'obligation d'envoyer `Content-Type: application/json` sur toutes les requêtes — un formulaire HTML externe ne peut pas forger une requête valide. L'API étant stateless (JWT), la protection CSRF classique par jeton de session ne s'applique pas.

4.4 Autres mesures

Mesure de sécurité
Mots de passe hachés avec bcrypt (coût 12)
Content Security Policy stricte (NelmioSecurityBundle) — <code>script-src 'self'</code>
En-têtes HTTP : X-Frame-Options, X-Content-Type-Options, Referrer-Policy

Mesure de sécurité
CORS restrictif — seule l'origine du front de production est autorisée
Secrets hors Git (.env.local) et injectés via variables Render
Upload d'images : types MIME validés (JPG/PNG/WebP), taille limitée
Tokens de réinitialisation à usage unique et expiration courte
Rate limiting sur login / register / forgot-password
ExceptionHandler : messages d'erreur génériques en production (pas de fuite)
Routes {id} contraintes à \d+ — matching déterministe des routes littérales

5. Déploiement

5.1 Frontend — Vercel

Paramètre	Valeur
Plateforme	Vercel — hébergement statique gratuit
Repository	github.com/NicolasVrignaudVG/Vite-Gourmand
Branche	main — déploiement auto à chaque push (webhook)
Build	Aucun build — HTML/CSS/JS servis directement
Routing SPA	vercel.json — proxy /api/* vers Render, puis catch-all vers index.html
Proxy API	Rewrites Vercel : le front appelle /api/* sur son propre domaine ; Vercel relaie vers l'API Render (cookies same-site)

5.2 Backend — Render (Docker)

Paramètre	Valeur
Plateforme	Render — Web Service Docker
Repository	github.com/NicolasVrignaudVG/Vite-Gourmand-back
Image	php:8.4-apache + extensions mongodb, pdo_mysql, intl
Port	10000 (attendu par le scan de ports Render)
Démarrage	assets:install puis apache2-foreground
Limites	Fichiers éphémères — images perdues au redéploiement

5.3 Bases de données

Service	Détail
MySQL (Clever Cloud)	MySQL 8.0 managé — charset utf8mb4, 5 connexions max (gratuit)
Migrations	Doctrine Migrations appliquées au déploiement
MongoDB Atlas	Cluster M0 gratuit — base vite_gourmand_stats
Collection stats	commandes — upsert à chaque commande (CA, volumes par menu)

5.4 Variables d'environnement (Render)

Variable	Description
APP_ENV	prod
APP_SECRET	Clé secrète Symfony
DATABASE_URL	URL MySQL Clever Cloud (identifiants masqués)
MONGO_URI / MONGO_DB	Connexion MongoDB Atlas + nom de base
MAILER_DSN	brevo+api://KEY@default
MAILER_SENDER_EMAIL	Adresse d'envoi (masquée)
CORS_ALLOW_ORIGIN	Origine du front autorisée
JWT_SECRET_KEY / JWT_PUBLIC_KEY	Clés RSA en base64
ORS_API_KEY	Clé API OpenRouteService

■ *Aucune valeur réelle de secret n'est reproduite dans ce document : identifiants de base, adresses e-mail et clés sont volontairement masqués.*

6. Structure du code & gestion de versions

6.1 Backend Symfony

Dossier	Rôle
src/Controller/	Contrôleurs API (Auth, Commande, Menu, Admin...)
src/Entity/	Entités Doctrine
src/Repository/	Requêtes personnalisées
src/Service/	Services métier (CommandeService, MailerService, MongoService...)
src/EventListener/	ExceptionListener — gestion JSON des erreurs
migrations/	Migrations Doctrine
config/jwt/	Clés RSA (non commitées)
Dockerfile	Image PHP 8.4 Apache pour Render

6.2 Frontend JavaScript

Fichier	Rôle
js/api.js	Fonctions d'appel API (Auth, Menus, Commandes, Admin)
js/script.js	Logique des pages, gestion d'état, rendu
js/router.js	Routeur SPA — chargement dynamique des fragments
css/main.css	Design tokens, composants, mise en page
pages/*.html	Fragments HTML des pages
index.html	Point d'entrée SPA
vercel.json	Rewrites : proxy /api/* vers Render + routing SPA

6.3 Stratégie de branches Git

Le projet suit un modèle à trois niveaux : main (production, déployée), develop (intégration), et des branches feature/* par fonctionnalité, mergées vers develop après test puis vers main.

Branche	Contenu
main	Code de production — déploiement automatique
develop	Intégration des fonctionnalités
feature/authentification	Connexion, inscription, JWT, reset password
feature/gestion-menus	CRUD menus, plats, images, thèmes, régimes
feature/commandes	Commande multi-menus, calcul livraison
feature/espace-utilisateur	Espace client — historique, profil
feature/espace-employe	Gestion commandes, avis, horaires
feature/espace-admin	Gestion employés, statistiques MongoDB